

Complete Guide to JavaScript 'this' Keyword

Table of Contents

1. What is 'this'?
 2. Global Context
 3. Regular Function Call
 4. Object Method
 5. Arrow Functions
 6. Constructor Functions
 7. Event Handlers
 8. Explicit Binding (call, apply, bind)
 9. setTimeout/setInterval
 10. Array Methods with Callbacks
 11. Method Assignment (Lost Context)
 12. Class Methods
 13. Nested Functions
 14. Callback Functions
 15. IIFE (Immediately Invoked Function Expression)
 16. Strict Mode vs Non-Strict Mode
-

What is 'this'?

The `this` keyword in JavaScript refers to the object that is currently executing the code. The value of `this` is determined by **how a function is called**, not where it's defined (except for arrow functions).

Key Rule: `this` is set at **call time**, not declaration time.

Scenario 1: Global Context

Code Example:

```
console.log(this);  
  
// Or in a block  
{  
  console.log(this);  
}
```

Output:

- Browser: `Window` object
- Node.js: `global` object

Explanation:

When code runs in the global scope (not inside any function), `this` refers to the global object. In browsers, this is the `window` object, and in Node.js, it's the `global` object.

Behind the Scenes:

JavaScript engine creates a global execution context → Sets `this` binding to the global object → Executes the code with `this` pointing to global object.

Scenario 2: Regular Function Call

Code Example:

```
function myFunction() {
  console.log(this);
  console.log(this === window); // true in browser (non-strict mode)
}

myFunction();
```

Output:

- Non-strict mode: `Window` object
- Strict mode: `undefined`
- Strict mode Syntax : `"use strict";`

Explanation:

- When a regular function is called without an explicit context (no object.method pattern), JavaScript applies default binding.
- In non-strict mode, `this` defaults to the global object. In strict mode, it becomes `undefined`.
- Strict mode removes the automatic binding of `this` to the global object.
- If a function is called in the default function call form (not as a method, not with `call`/`apply`, etc.), `this` is set to `undefined` instead of `window`.
- Historically, JavaScript allowed defaulting `this` to the global object (for convenience). But this caused security bugs & unexpected behavior (functions unintentionally polluting or modifying global state).
- Strict mode was introduced in ES5 (ECMAScript 2009) to make behavior safer and more predictable.

Behind the Scenes:

Function called without context → JavaScript checks if strict mode → If non-strict: `this = global object`
 → If strict: `this = undefined`

Scenario 3: Object Method

Code Example:

```
const person = {
  name: 'Alice',
  age: 30,
  greet: function() {
    console.log(this.name); // 'Alice'
    console.log(this.age);  // 30
    console.log(this);      // person object
  }
};
person.greet();
```

Output:

```
Alice
30
{name: 'Alice', age: 30, greet: f}
```

Explanation:

When a function is called as a method of an object (using dot notation), `this` refers to the object that owns the method. The object to the left of the dot becomes `this`.

Behind the Scenes:

`person.greet()` called → JavaScript sees `object.method()` pattern → Applies implicit binding → Sets `this` to the object (`person`) → Method executes with `this` pointing to `person`

Scenario 4: Arrow Functions

```
const globalArrow = () => {
  console.log(this); // Window object
};
const obj = {
  name: 'Bob',
  regularMethod: function() {
    console.log('Regular:', this.name); // 'Bob'
    const arrowInside = () => {
      console.log('Arrow inside:', this.name); // 'Bob'
    };
    arrowInside();
  },
  arrowMethod: () => {
    console.log('Arrow method:', this.name); // undefined
  }
};
globalArrow();
obj.regularMethod();
obj.arrowMethod();
```

Output:

```
Window object
Regular: Bob
Arrow inside: Bob
Arrow method: undefined
```

Explanation:

Arrow functions use **lexical `this` binding** - they inherit `this` from their enclosing scope at the time they're created. They don't have their own `this` and cannot be bound to a different context.

Behind the Scenes:

Arrow function created → Captures `this` value from surrounding scope → Stores this binding permanently → When called, always uses the captured `this` value, regardless of call context

Scenario 5: Constructor Functions

Code Example:

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    console.log(`Hi, I'm ${this.name}`);
  };
  console.log(this); // New Person instance
}

const person1 = new Person('Charlie', 25);
const person2 = new Person('Diana', 28);

person1.greet(); // "Hi, I'm Charlie"
person2.greet(); // "Hi, I'm Diana"
```

Output:

```
Person {name: 'Charlie', age: 25, greet: f}
Person {name: 'Diana', age: 28, greet: f}
Hi, I'm Charlie
Hi, I'm Diana
```

Explanation:

When a function is called with the `new` keyword, `this` refers to the newly created object instance. The constructor function sets up properties and methods on this new object.

Behind the Scenes:

`new Person()` called → JavaScript creates empty object → Sets `this` to the new object → Executes constructor function → Returns the new object (unless constructor explicitly returns something else)

Scenario 6: Event Handlers

Code Example:

```
const button = document.querySelector('button');

// Regular function as event handler
button.onclick = function() {
  console.log(this); // The button element
  console.log(this.textContent); // Button's text
};

// Arrow function as event handler
button.addEventListener('click', () => {
  console.log(this); // Window object (not the button!)
});

// Method as event handler
const handler = {
  handleClick: function() {
    console.log(this); // The button element (when assigned to onclick)
  }
};

button.onclick = handler.handleClick;
```

Explanation:

In event handlers, `this` typically refers to the element that triggered the event when using regular functions. Arrow functions maintain their lexical binding and don't point to the element.

Behind the Scenes:

Event triggered → Browser calls the handler function → Sets `this` to the target element (for regular functions) → Arrow functions ignore this and use their lexical `this`

Scenario 7: Explicit Binding (call, apply, bind)

Code Example:

```
function greet(greeting, punctuation) {  
  console.log(`${greeting}, I'm ${this.name}${punctuation}`);  
}  
  
const person1 = { name: 'Eve' };  
const person2 = { name: 'Frank' };  
  
// call() - arguments passed individually  
greet.call(person1, 'Hello', '!'); // "Hello, I'm Eve!"  
  
// apply() - arguments passed as array  
greet.apply(person2, ['Hi', '.']); // "Hi, I'm Frank."  
  
// bind() - creates new function with bound 'this'  
const boundGreet = greet.bind(person1);  
boundGreet('Hey', '?'); // "Hey, I'm Eve?"  
  
// bind() with partial application  
const boundGreetWithHello = greet.bind(person2, 'Hello');  
boundGreetWithHello('!!!'); // "Hello, I'm Frank!!!"
```

Output:

```
Hello, I'm Eve!  
Hi, I'm Frank.  
Hey, I'm Eve?  
Hello, I'm Frank!!!
```

Explanation:

- `call()`: Calls function immediately with specified `this` and arguments
- `apply()`: Same as call, but arguments passed as array
- `bind()`: Creates new function with permanently bound `this` (and optionally some arguments)

Behind the Scenes:

Explicit binding methods override JavaScript's normal `this` binding rules → Function executes with the explicitly provided `this` value → `bind()` creates a new function wrapper that always uses the bound context

Scenario 8: setTimeout/setInterval

Code Example:

```
const obj = {
  name: 'Grace',
  delayedGreet: function() {
    console.log('Immediate:', this.name); // 'Grace'

    setTimeout(function() {
      console.log('Timeout regular:', this.name); // undefined
    }, 1000);

    setTimeout(() => {
      console.log('Timeout arrow:', this.name); // 'Grace'
    }, 1000);

    // Solution with bind
    setTimeout(function() {
      console.log('Timeout bound:', this.name); // 'Grace'
    }.bind(this), 1000);
  }
};

obj.delayedGreet();
```

Output:

```
Immediate: Grace
Timeout regular: undefined
Timeout arrow: Grace
Timeout bound: Grace
```

Explanation:

`setTimeout` and `setInterval` call their callback functions in the global context, so `this` becomes the global object. Arrow functions and `bind()` can preserve the original context.

Behind the Scenes:

`setTimeout` queues callback → When timer expires, callback executed in global context → Regular functions get global `this` → Arrow functions keep their lexical `this` → `bind()` creates wrapper with fixed `this`

Scenario 9: Array Methods with Callbacks

Code Example:

```
const numbers = [1, 2, 3, 4, 5];
const context = { multiplier: 10 };

// Regular callback - 'this' is global
numbers.forEach(function(num) {
  console.log(this); // Window object
  console.log(num * (this.multiplier || 1)); // num * 1 (undefined multiplier)
});

// Arrow callback - 'this' is lexical
numbers.forEach((num) => {
  console.log(this); // Window object (if in global scope)
  console.log(num * (this.multiplier || 1));
});

// Providing 'this' context as second argument
numbers.forEach(function(num) {
  console.log(this.multiplier); // 10
  console.log(num * this.multiplier); // num * 10
}, context);

// Using bind
numbers.forEach(function(num) {
  console.log(num * this.multiplier); // num * 10
}.bind(context));
```

Explanation:

Array methods like `forEach`, `map`, `filter` call their callbacks in global context by default. Some methods accept a second parameter to specify `this`, or you can use `bind()` or arrow functions.

Behind the Scenes:

Array method iterates → Calls callback for each element → Default `this` is global object → If `thisArg` provided, callback called with that as `this` → Arrow functions ignore `thisArg` and use lexical `this`

Scenario 10: Method Assignment (Lost Context)

Code Example:

```
const person = {
  name: 'Henry',
  greet: function() {
    console.log(`Hello, I'm ${this.name}`);
  }
};

// Direct method call - works fine
person.greet(); // "Hello, I'm Henry"

// Method assignment - loses context
const greetFunction = person.greet;
greetFunction(); // "Hello, I'm undefined"

// Solutions:
// 1. Using bind
const boundGreet = person.greet.bind(person);
boundGreet(); // "Hello, I'm Henry"

// 2. Using arrow function wrapper
const wrappedGreet = () => person.greet();
wrappedGreet(); // "Hello, I'm Henry"

// 3. Using call
greetFunction.call(person); // "Hello, I'm Henry"
```

Output:

```
Hello, I'm Henry
Hello, I'm undefined
Hello, I'm Henry
Hello, I'm Henry
Hello, I'm Henry
```

Explanation:

When you assign a method to a variable, it loses its connection to the original object. The function becomes a regular function call, so `this` defaults to global object.

Behind the Scenes:

Method assigned to variable → Reference copied without object context → When called, no implicit binding occurs → `this` falls back to default binding (global object)

Scenario 11: Class Methods

Code Example:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }

  // Arrow method (experimental syntax)
  arrowSpeak = () => {
    console.log(`${this.name} makes a sound (arrow)`);
  }
}

const cat = new Animal('Whiskers');
const dog = new Animal('Buddy');

// Normal method calls
cat.speak(); // "Whiskers makes a sound"
dog.speak(); // "Buddy makes a sound"

// Method assignment issue
const speakFunction = cat.speak;
speakFunction(); // Error in strict mode or "undefined makes a sound"

// Arrow method preserves context
const arrowSpeakFunction = cat.arrowSpeak;
arrowSpeakFunction(); // "Whiskers makes a sound (arrow)"
```

Explanation:

Class methods work like object methods. Arrow methods (class fields) bind `this` to the instance when created, preventing context loss during assignment.

Behind the Scenes:

Class methods are added to prototype → Instance methods have `this` bound at call time → Arrow methods are created as instance properties with fixed `this` binding → Classes run in strict mode by default

Scenario 12: Nested Functions

Code Example:

```
const obj = {
  name: 'Iris',
  outerMethod: function() {
    console.log('Outer this:', this.name); // 'Iris'

    function innerFunction() {
      console.log('Inner this:', this.name); // undefined (strict mode)
    }
    innerFunction();

    const innerArrow = () => {
      console.log('Inner arrow this:', this.name); // 'Iris'
    };
    innerArrow();

    // Solution: save reference to this
    const self = this;
    function innerWithSelf() {
      console.log('Inner with self:', self.name); // 'Iris'
    }
    innerWithSelf();
  }
};

obj.outerMethod();
```

Output:

```
Outer this: Iris
Inner this: undefined
Inner arrow this: Iris
Inner with self: Iris
```

Explanation:

Nested regular functions lose the outer `this` context and default to global binding. Arrow functions inherit `this` from their enclosing scope. Common solutions include arrow functions, `bind()`, or saving `this` to a variable.

Behind the Scenes:

Outer method called with object context → Inner regular function called without context → Gets default binding (global/undefined) → Arrow function captures `this` from outer scope → Variable assignment preserves outer `this`

Scenario 13: Callback Functions

Code Example:

```
const user = {
  name: 'Jack',
  processData: function(callback) {
    console.log('Processing...');
    callback();
  },

  getData: function() {
    console.log(`Getting data for ${this.name}`);
  }
};

// Lost context in callback
user.processData(user.getData); // "Getting data for undefined"

// Solutions:
// 1. Arrow function wrapper
user.processData(() => user.getData()); // "Getting data for Jack"

// 2. Bind method
user.processData(user.getData.bind(user)); // "Getting data for Jack"

// 3. Call with specific context
function processWithContext(callback, context) {
  callback.call(context);
}
processWithContext(user.getData, user); // "Getting data for Jack"
```

Explanation:

When passing methods as callbacks, they lose their original context. The callback is called as a regular function, not as a method of the original object.

Behind the Scenes:

Method passed as callback → Reference passed without object binding → Callback invoked in different context → `this` becomes global object → Solutions restore original context through various binding techniques

Scenario 14: IIFE (Immediately Invoked Function Expression)

Code Example:

```
const obj = {
  name: 'Kate',
  test: function() {
    console.log('Method this:', this.name); // 'Kate'

    // IIFE with regular function
    (function() {
      console.log('IIFE this:', this.name); // undefined
   })();

    // IIFE with arrow function
    (() => {
      console.log('IIFE arrow this:', this.name); // 'Kate'
   })();

    // IIFE with explicit binding
    (function() {
      console.log('IIFE bound this:', this.name); // 'Kate'
    }).call(this);
  }
};

obj.test();
```

Output:

```
Method this: Kate
IIFE this: undefined
IIFE arrow this: Kate
IIFE bound this: Kate
```

Explanation:

IIFEs are regular function calls, so they follow normal `this` binding rules. Regular IIFEs get global `this`, arrow IIFEs inherit from enclosing scope, and you can explicitly bind `this`.

Behind the Scenes:

IIFE created and immediately invoked → Regular IIFE called without context → Gets default binding → Arrow IIFE inherits `this` → `.call(this)` explicitly sets `this`

Scenario 15: Strict Mode vs Non-Strict Mode

Code Example:

```
// Non-strict mode
function nonStrictFunction() {
  console.log('Non-strict this:', this); // Window object
  console.log('this === window:', this === window); // true
}

// Strict mode
function strictFunction() {
  'use strict';
  console.log('Strict this:', this); // undefined
  console.log('this === undefined:', this === undefined); // true
}

nonStrictFunction();
strictFunction();

// In objects, both behave the same
const obj = {
  nonStrict: function() {
    console.log('Object non-strict:', this === obj); // true
  },
  strict: function() {
    'use strict';
    console.log('Object strict:', this === obj); // true
  }
};

obj.nonStrict();
obj.strict();
```

Output:

```
Non-strict this: Window
this === window: true
Strict this: undefined
this === undefined: true
Object non-strict: true
Object strict: true
```

Explanation:

In strict mode, `this` is not automatically bound to the global object in function calls. It remains `undefined` unless explicitly set. This helps prevent accidental global variable creation.

Behind the Scenes:

Strict mode changes default binding behavior → Instead of global object fallback, `this` stays `undefined` → Object method calls still work the same → Helps catch errors where `this` is unexpectedly global

Summary of 'this' Binding Rules

1. **Default Binding:** Global object (non-strict) or `undefined` (strict)
2. **Implicit Binding:** Object that owns the method
3. **Explicit Binding:** Object specified with `call()`, `apply()`, or `bind()`
4. **Arrow Functions:** Lexical binding from enclosing scope
5. **Constructor:** New object being created
6. **Event Handlers:** Element that triggered the event

Common Pitfalls and Solutions

Problem	Solution
Method assignment loses context	Use <code>bind()</code> , arrow functions, or wrapper functions
Callbacks lose context	Arrow functions, <code>bind()</code> , or save <code>this</code> to variable
<code>setTimeout</code> loses context	Arrow functions or <code>bind()</code>
Nested functions lose context	Arrow functions, <code>bind()</code> , or save <code>this</code> reference
Event handlers with arrow functions	Use regular functions to get element reference

Best Practices

1. Use arrow functions for callbacks to preserve context
2. Be explicit with `bind()` when passing methods as callbacks
3. Avoid relying on default binding - it's error-prone
4. Use strict mode to catch `this` binding errors early
5. Consider class arrow methods for methods that will be passed around
6. Save `this` to a variable (`const self = this`) when needed in nested functions